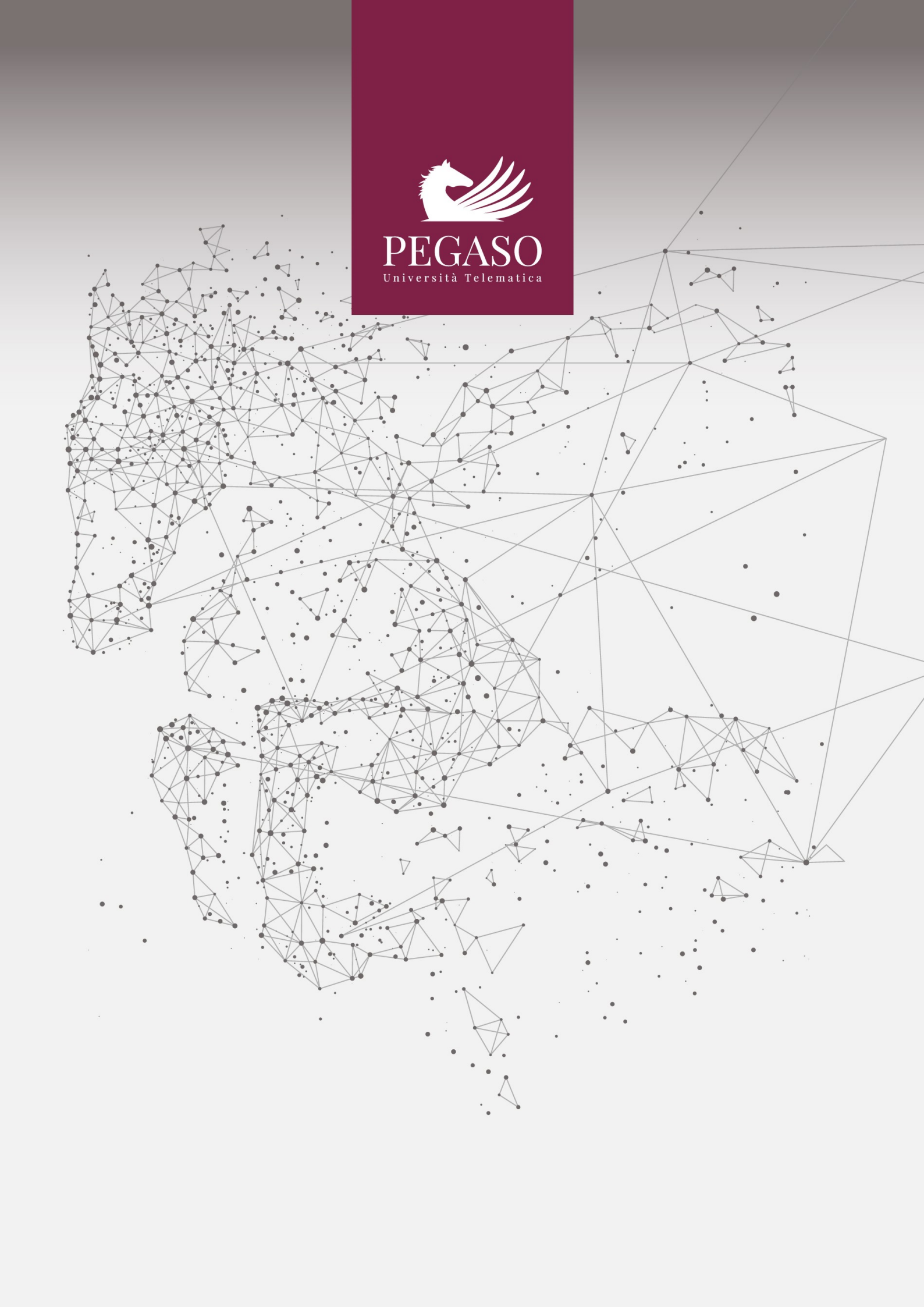

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ARCHITETTURE BASATE SU OGGETTI E SERVIZI 3

2. ARCHITETTURE BASATE SU RISORSE 6

3. ARCHITETTURE PUBLISH-SUBSCRIBE 8

BIBLIOGRAFIA 11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Architetture basate su oggetti e servizi

Le architetture basate su oggetti, come quella illustrata nella Figura 1, aderiscono a uno schema organizzativo molto più libero rispetto alle architetture stratificate. Ogni oggetto corrisponde ad un componente e ciascuno di questi componenti è collegato tramite un meccanismo noto come chiamata di procedura. In un contesto di sistema distribuito, è possibile che una chiamata di procedura avvenga su una rete. Ciò significa che l'oggetto che esegue la chiamata non deve necessariamente essere in esecuzione sullo stesso computer dell'oggetto chiamato.

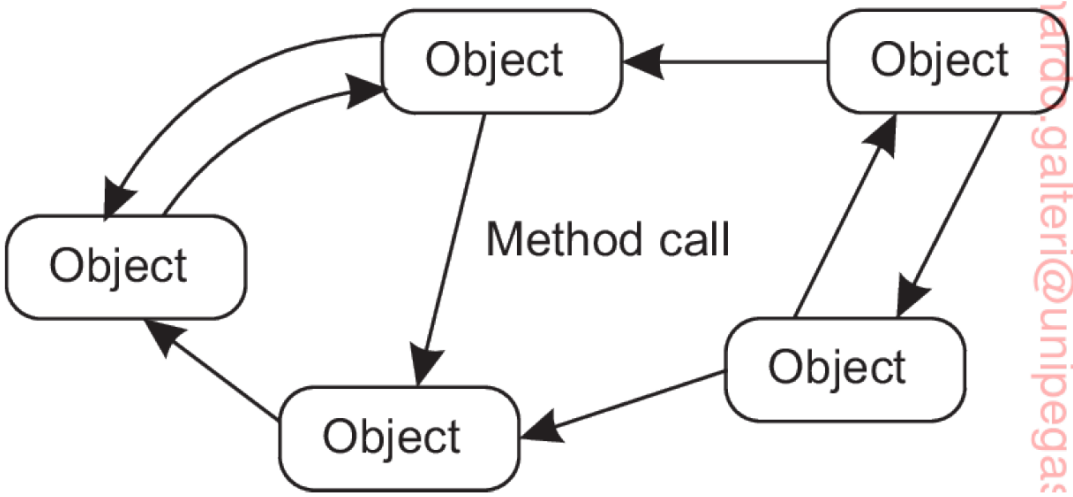


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Le architetture basate su oggetti sono interessanti per il motivo che offrono un modo naturale per incapsulare i dati (che viene indicato come stato di un oggetto) e le operazioni che possono essere eseguite su tali dati (che sono indicati come metodi di un oggetto) in una singola entità. Poiché i dettagli dell'implementazione sono nascosti dall'interfaccia che un oggetto presenta ai suoi utenti, possiamo, in teoria, pensare a un oggetto come una entità completamente indipendente dall'ambiente in cui opera. Come nel caso dei componenti, ciò indica che un oggetto dovrebbe essere sostituibile con un altro che possiede un'interfaccia identica sotto ogni altro aspetto se la sua interfaccia è correttamente definita e gli altri aspetti del suo design non vengono alterati.

A causa del fatto che le interfacce sono distinte dagli oggetti che implementano queste interfacce, siamo in grado di posizionare un'interfaccia su una macchina mentre l'oggetto stesso risiede su un'altra

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

macchina. Questo tipo di organizzazione, che può essere visto nella Figura 2, è ciò che le persone di solito intendono quando parlano di un oggetto distribuito.

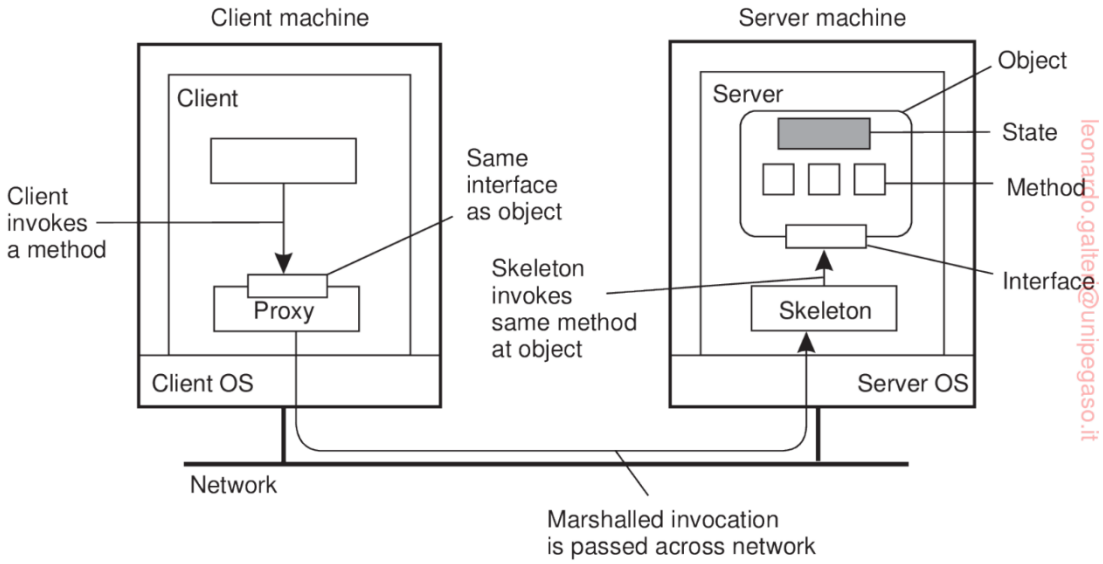


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Quando un client esegue il binding a un oggetto distribuito, un'implementazione dell'interfaccia dell'oggetto, nota come proxy, viene caricata nello spazio degli indirizzi del client. Quando si tratta di sistemi basati su RPC, un proxy può essere paragonato a un client stub. Il suo unico scopo è convertire le chiamate del metodo in messaggi ed eseguire l'unmarshalling dei messaggi di risposta prima di passare le informazioni al client. L'oggetto effettivo risiede su una macchina server, il quale espone la stessa interfaccia che ha sulla macchina cliente. Innanzitutto, le richieste di chiamata in ingresso vengono inviate a un server stub, che esegue l'unmarshalling delle richieste prima di utilizzarle per chiamare i metodi sull'interfaccia dell'oggetto sul server. Inoltre, è responsabilità del server stub eseguire il marshalling delle risposte e inoltrare i messaggi di risposta al proxy sul lato client.

Poiché è l'unica cosa che consente al middleware del server di accedere agli oggetti definiti dall'utente, uno stub lato server viene spesso definito scheletro. In pratica, spesso contiene codice incompleto sotto forma di una classe specifica del linguaggio che lo sviluppatore deve modificare ulteriormente per soddisfare le proprie esigenze specifiche.

Il fatto che lo stato della maggior parte degli oggetti distribuiti non sia distribuito e risieda invece su una singola macchina è una caratteristica di questi oggetti, anche se sembra in qualche modo controintuitivo. Le uniche interfacce rese disponibili su altri computer sono quelle implementate dall'oggetto. Questo tipo di entità sono a volte indicate come "oggetti remoti". È possibile che lo stato di un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

oggetto sia distribuito su più macchine, ma il fatto che questo sia effettivamente distribuito è nascosto ai client dalle interfacce dell'oggetto.

È possibile sostenere che le architetture basate su oggetti sono gli elementi fondamentali per racchiudere i servizi all'interno di unità autonome. Incapsulamento è il termine che meglio descrive questa situazione; il servizio nella sua interezza è sviluppato come entità indipendente, nonostante il fatto che possa avvalersi di servizi aggiuntivi. Stiamo quindi aprendo la strada verso architetture orientate ai servizi, più comunemente indicate come Service-Oriented Architectures (SOA), separando chiaramente i vari servizi in modo tale che possano operare indipendentemente l'uno dall'altro.

Un'applicazione o un sistema distribuito viene essenzialmente assemblato come una composizione di molti servizi diversi quando si utilizza un'architettura orientata alla fornitura di servizi. È possibile che non tutti questi servizi rientrino nell'ambito di competenza dello stesso organo amministrativo. Quando si parla di cloud computing, è possibile che un'organizzazione che esegue la propria applicazione aziendale utilizzi i servizi di storage offerti da un cloud provider. I clienti hanno accesso a un'interfaccia per questi servizi di archiviazione, che sono logicamente combinati in una singola unità per comodità.

Anche se lo storage è un servizio relativamente di base, è facile pensare a scenari più complessi che potrebbero trarne beneficio. Prendiamo, ad esempio, un sito web che vende beni digitali come gli e-book come esempio. Un'applicazione che elabora gli ordini e a sua volta opera su un database locale che memorizza gli e-book, è un esempio di implementazione semplice che aderisce alla stratificazione dell'applicazione. Questo tipo di implementazione può essere abbastanza semplice. L'elaborazione degli ordini comporta in genere la selezione degli articoli, la registrazione e il controllo del canale di consegna (possibilmente utilizzando la posta elettronica), oltre a garantire che un pagamento venga elaborato correttamente. Quest'ultimo può essere curato da un servizio separato gestito da un'organizzazione diversa. Un cliente che sta acquistando un e-book verrà indirizzato lì per il pagamento, quindi l'organizzazione che vende l'e-book verrà avvisata in modo che possa completare la transazione.

Se visto in questa luce, possiamo vedere che la sfida di sviluppare un sistema distribuito è principalmente una questione di composizione del servizio, in particolare garantendo che i vari componenti del sistema lavorino insieme in modo efficace. In effetti, i problemi relativi all'integrazione delle applicazioni aziendali sono estremamente simili a questo particolare problema. È fondamentale che ogni servizio fornisca un'interfaccia di programmazione chiaramente delineata. In realtà, ciò significa anche che ogni servizio offre la propria interfaccia, che, a sua volta, può rendere la composizione dei servizi significativamente più complessa di quanto non sarebbe altrimenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Architetture basate su risorse

I ricercatori hanno iniziato a riconsiderare l'architettura dei sistemi distribuiti basati sul Web man mano che un numero crescente di servizi diventava disponibile sul Web e lo sviluppo di sistemi distribuiti attraverso la composizione dei servizi diventava più importante. La connessione dei diversi componenti di un servizio può diventare rapidamente un'attività di integrazione difficile e dispendiosa in termini di tempo, che è uno dei problemi con la composizione del servizio.

Un altro modo di considerare un sistema distribuito è come una grande raccolta di risorse gestite individualmente da diversi componenti. Questo è un modo di guardare ai sistemi distribuiti da un altro punto di vista. Le applicazioni, sia locali che remote, hanno la possibilità di aggiungere o rimuovere risorse, nonché di recuperare o modificare quelle esistenti. Questa strategia, al momento ampiamente utilizzata per il World Wide Web e nota come Representational State Transfer (REST), è stata introdotta per la prima volta. Le architetture RESTful si distinguono per le seguenti quattro caratteristiche chiave:

1. Una convenzione di naming unificata viene utilizzata per tutti gli identificatori di risorsa.
2. Ciascuno dei servizi fornisce la stessa interfaccia utente, che comprende non più di quattro diverse operazioni.
3. I messaggi diretti o provenienti da un servizio sono completamente in grado di descrivere sé stessi.
4. Una volta completata un'operazione presso un servizio, quel componente dimenticherà tutto della persona che ha effettuato la chiamata.

   La caratteristica finale è anche nota come esecuzione stateless.

L'utilizzo di un servizio del cloud storage Simple Storage Service (S3) di Amazon, come esempio di come RESTful può funzionare in pratica, è un buon modo per illustrare questo concetto. Amazon S3 supporta solo due risorse: oggetti, che possono essere pensati come l'equivalente di file, e bucket, che possono essere pensati come l'equivalente di directory. La modalità in cui i bucket possano essere posizionati all'interno di altri bucket non è prevista. Un oggetto con il nome ObjectName memorizzato nel bucket BucketName è riferito dall'URI (Uniform Resource Identifier):

http://BucketName.s3.amazonaws.com/ObjectName.

Un'applicazione, in sostanza, invia una richiesta PUT con l'URI del bucket o dell'oggetto per creare un bucket. Questo avverrebbe anche nel caso in cui si volesse creare un oggetto. In generale, il protocollo HTTP è quello utilizzato quando si lavora con questo tipo di servizio. In altre parole, è semplicemente un'altra richiesta HTTP, che S3, alla fine, interpreterà correttamente. Se esiste già un'istanza dell'oggetto o del bucket, verrà restituito un messaggio di errore HTTP.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Analogamente, affinché un'applicazione possa apprendere quali oggetti sono archiviati in un bucket, l'URI di tale bucket deve essere incluso in una richiesta GET inviata dall'applicazione. Sempre sotto forma di risposta HTTP standard, S3 fornirà un elenco dei nomi degli oggetti memorizzati.

Grazie alla sua semplicità, l'architettura RESTful ha recentemente ricevuto molta attenzione. La questione se i servizi RESTful siano o meno superiori a quelli in cui i servizi sono specificati mediante interfacce specifiche del servizio è attualmente oggetto di acceso dibattito. I due metodi sono stati confrontati in dettaglio, e i risultati hanno mostrato che, come era prevedibile, ogni metodo possiede sia aspetti positivi che negativi. In particolare, la facilità d'uso fornita dalle architetture RESTful può rendere difficile trovare soluzioni semplici a schemi di comunicazione complessi. Uno di questi scenari è quello che richiede transazioni distribuite che, come regola generale, necessita di servizi che monitorano lo stato di esecuzione corrente. D'altra parte, ci sono molti esempi in cui le architetture RESTful si adattano perfettamente a un semplice schema di integrazione dei servizi, tuttavia dove la miriade di interfacce dei servizi complicherà le cose.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Architetture Publish-subscribe

Poiché le dimensioni dei sistemi continuano ad espandersi e nuovi processi possono essere aggiunti o rimossi con maggiore facilità, sta diventando sempre più importante disporre di un'architettura che consenta alle dipendenze tra i processi di diventare il meno strette possibile. Un numero significativo di sistemi distribuiti ha optato per implementare un sistema architetturale che presenta una chiara divisione tra le funzioni di elaborazione e coordinamento. Il modo migliore per comprendere un sistema è pensarlo come un insieme di processi che vengono eseguiti indipendentemente. Nel contesto di questo modello, il coordinamento si riferisce alla comunicazione e alla collaborazione che avviene tra i processi. Agisce come il collante che lega insieme tutte le diverse attività svolte dai processi.

Esiste una tassonomia dei modelli di coordinamento che è applicabile in modo equivalente a un'ampia varietà di configurazioni di sistemi distribuiti. Si distinguono modelli considerando due dimensioni distinte, vale a dire temporale e referenziale.

Il coordinamento diretto avviene ogni volta che c'è un accoppiamento temporale e referenziale tra due processi. Nella maggior parte dei casi, l'accoppiamento referenziale si manifesta sotto forma di riferimento esplicito nel processo di comunicazione. Ad esempio, affinché un processo possa comunicare, deve prima essere a conoscenza del nome o dell'identificatore degli altri processi con cui desidera condurre lo scambio di informazioni. A causa dell'accoppiamento temporale, è necessario che due processi comunicanti siano attivi e funzionino correttamente. La comunicazione diretta può essere vista in azione quando le persone parlano tra loro usando i loro telefoni cellulari (presumendo che ogni telefono cellulare sia di proprietà di una singola persona).

Il coordinamento che avviene quando i processi sono temporalmente disaccoppiati ma accoppiati da un punto di vista referenziale viene definito "mailbox coordination" ed è un tipo di coordinamento completamente diverso dal precedente. In questa particolare circostanza, non è necessario che vi sia l'esecuzione simultanea di due processi comunicanti affinché la comunicazione abbia luogo. Infatti, la comunicazione viene effettuata depositando i messaggi in una casella di posta che può essere oppure può non essere condivisa. C'è un accoppiamento referenziale perché è necessario indirizzare esplicitamente la cassetta postale che conterrà i messaggi che devono essere scambiati.

Il gruppo di modelli per il coordinamento basato sugli eventi è formato dalla combinazione di sistemi che sono referenzialmente disaccoppiati e temporalmente accoppiati tra loro. Nei sistemi con processi disaccoppiati referenzialmente, tali processi non si conoscono esplicitamente. L'unica azione che un processo è in grado di intraprendere è quella di trasmettere una notifica che dettaglia il verificarsi di un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

evento (ad esempio, che vuole coordinare le attività o che ha appena prodotto alcuni risultati interessanti). Supponendo che le notifiche siano disponibili in un'ampia varietà di forme, i processi hanno la capacità di sottoscrivere una particolare notifica. In un modello perfetto per il coordinamento basato sugli eventi, una notifica che è stata pubblicata verrà inviata direttamente ai processi che si sono iscritti per riceverla. Tuttavia, nella maggior parte dei casi, è necessario che il processo iscritto sia operativo nel momento in cui la notifica è stata messa a disposizione pubblicamente.

La combinazione di processi che sono disaccoppiati referenzialmente e temporalmente, che si traduce in quello che è noto come uno spazio dati condiviso, costituisce un modello ben noto per il coordinamento delle attività. Le tuple, che sono record di dati strutturati costituiti da un numero di campi e sono molto simili a una riga in una tabella di database, sono il mezzo principale con cui i processi si scambiano informazioni tra loro. Qualsiasi tipo di tupla può essere memorizzata nello spazio dati condiviso se il processo lo utilizza. Un processo deve prima fornire un modello di ricerca, che viene quindi valutato rispetto alle tuple per recuperare la tupla desiderata. Qualsiasi tupla che soddisfi i criteri verrà restituita.

Da ciò si può dedurre che un meccanismo di ricerca associativa per le tuple è implementato utilizzando spazi di dati condivisi. Per estrarre una tupla dal dataspace, un processo deve prima specificare (almeno alcuni dei) valori dei campi a cui è interessato. Successivamente, qualsiasi tupla che soddisfa la specifica di cui sopra viene estratta dallo spazio dati e trasmessa al processo. Un processo può sottoscrivere determinate tuple fornendo un modello di ricerca; quando un processo inserisce una tupla nello spazio dati, i sottoscrittori corrispondenti ricevono una notifica.

Gli spazi di dati condivisi sono spesso combinati con il coordinamento basato su eventi. In entrambi questi scenari, abbiamo a che fare con un'architettura di pubblicazione-sottoscrizione (publish-subscribe) e la caratteristica distintiva principale di questo tipo di architettura è che i singoli processi non fanno alcun riferimento diretto l'uno all'altro.

La comunicazione avviene nei sistemi di pubblicazione-sottoscrizione descrivendo gli eventi di cui un abbonato è interessato, che è una caratteristica essenziale di questo tipo di sistemi. Di conseguenza, il naming gioca un ruolo cruciale.

Si assuma che gli eventi possano essere caratterizzati da un elenco di attributi. Quando una notifica che descrive un evento viene resa disponibile per la lettura di altri processi, si parla di "pubblicazione" della notifica. A tale scopo, è necessario inviare una richiesta iscrizione al middleware. All'interno di questa iscrizione, deve esserci una descrizione dell'evento a cui l'iscritto è interessato. Questo tipo di descrizione consiste tipicamente di diverse coppie (attributo, valore), che è tipico per i cosiddetti sistemi di pubblicazione-sottoscrizione basati su argomenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nei sistemi di pubblicazione-sottoscrizione basati sul contenuto, una sottoscrizione può anche essere costituita da coppie di attributi-intervallo come alternativa aggiuntiva. In questo particolare scenario, si prevede che il valore dell'attributo specificato rientri in un determinato intervallo. A volte, le descrizioni possono essere fornite utilizzando un'ampia varietà di predicati formulati sugli attributi. La natura di questi predicati è molto simile a quella delle query di tipo SQL nel caso di database relazionali. È ovvio che più una descrizione è intricata, più difficile sarà determinare se un'occorrenza corrisponde o meno a una descrizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Pautasso C., Zimmermann O., and Leymann F. Restful Web Services vs. "Big" Web Services: Making the Right Architectural Decision. In 17th International World Wide Web Conference, 2008.
- Mühl G., Fiege L., and Pietzuch P. Distributed Event-Based Systems. Springer-Verlag, Berlin, 2006.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).